

In-Situ Cache: A Scalable and Efficient Configurable Scratchpad/Cache Memory for Tightly Coupled Multicore Clusters

Diyou Shen¹, Zexin Fu¹, Chi Zhang¹, Yichao Zhang¹, Bowen Wang¹, Angelo Garofalo^{1,2}, and Luca Benini^{1,2}

¹ETH Zürich, Zürich, Switzerland, ²Università di Bologna, Bologna, Italy
{dishen, zexifu, chizhang, yiczhang, bowwang, agarofalo, lbenini}@iis.ee.ethz.ch

Abstract—Modern edge AI systems often employ Scratchpad Memories (SPMs) for their low hardware cost and energy efficiency. However, SPMs’ reliance on software-managed data orchestration makes them hard to program and/or inefficient for workloads with irregular, data-dependent memory access patterns. State-of-the-art generative AI models mix regular, predictable accesses to dense tensors with data-dependent accesses to sparse, quantized tensors. This motivates the design of tightly coupled clusters of parallel processing elements (PEs) with configurable SPM/cache L1 memory capable of handling multiple parallel accesses.

We present In-Situ Cache, a low-cost, scalable, non-blocking cache architecture that eliminates dedicated miss handling storage by repurposing underutilized cache lines to store miss states in place, providing scalable miss-handling capacity. We build a hybrid SPM/cache design that enables runtime-configurable dual-mode access to a multi-banked SRAM memory. We integrate it as a 128 KiB four-way L1 cache into an open-source RISC-V vector compute cluster with eight 64-bit Floating Point Units (FPUs), originally based on a 128 KiB SPM-only L1 memory. Implemented in TSMC 7 nm technology, the proposed design operates at 1 GHz, matching the maximum frequency of the baseline cluster, with only 8% area overhead from the cache controller. On dense, regular workloads, In-Situ Cache configured in SPM-mode achieves at-roofline performance at 15.76 GFLOPS and 102 GFLOPS/W with negligible power overhead from its extended cache features. On sparse, irregular workloads, In-Situ Cache configured in cache-mode delivers up to $1.43\times$ speedup over an optimized double-buffered SPM baseline on sparse attention and vector-quantized GEMM workloads, while reducing programming effort.

Index Terms—RISC-V, Non-Blocking Cache, Miss Handling Architecture, Vector Processor.

I. INTRODUCTION

Modern edge AI systems increasingly adopt heterogeneous clusters built around Scratchpad Memory (SPM)-based memory hierarchies, combined with Direct Memory Access (DMA) engines that enable overlapping data movement with computation through software-managed double buffering [1]–[3]. This paradigm delivers high energy efficiency and effectively hides memory latency that ranges from tens of nanoseconds in tightly integrated systems to thousands of nanoseconds in disaggregated platforms [4], [5].

However, software-managed orchestration breaks down for the irregular, data-dependent access patterns increasingly common in

modern AI: the working set often exceeds SPM capacity, runtime-determined access patterns undermine the efficiency of double buffering, and concurrent irregular accesses in heterogeneous clusters from multiple cores and accelerators further amplify unpredictability. Data-dependent memory access patterns are becoming increasingly common in state-of-the-art models. Vector dequantization requires indirect codebook lookups for tensor reconstruction [6], sparse attention involves input-dependent indexing [7], and sparse tensor contractions rely on data-driven gather/scatter over large address ranges.

Non-blocking caches offer a natural complement to SPMs: they handle irregular accesses transparently, while still enabling local data reuse, and help curtailing off-chip traffic through miss merging. Recognizing this, modern architectures increasingly rely on a *hybrid* SPM/cache memory subsystem that combines the energy efficiency of software-managed SPM for regular workloads with the flexibility of caching for irregular ones. Notable examples include NVIDIA’s GPUs [8] and many-core AI accelerators such as the ET-SoC-1 [9]. However, the space cost of non-blocking caches is a concern, hindering their adoption for area-constrained edge AI accelerators. Moreover, to absorb the increased miss traffic from multiple concurrently executing processing elements, a larger number of in-flight requests must be tracked simultaneously [10], demanding proportionally large Miss Handling Architectures (MHAs). However, traditional MHAs scale poorly: at a modest 32-entry baseline, the miss handling storage accounts for only around 5% of the controller area, but expanding it eightfold causes the miss handling storage alone to exceed the entire controller size in both area and power [11]. In a hybrid design, this overhead directly competes with the shared SRAM budget, undermining the efficiency that motivates the hybrid approach.

A closer look at cache utilization reveals an underexploited opportunity. A significant fraction of cache capacity is occupied by lines that are never reused before eviction [12]–[14]: Jorge et al. [13] report that only about 25% of cache lines are actively used, while Konstantinos et al. [14] observe that, on average, only 8% of Last Level Cache (LLC) lines are referenced more than once in data-intensive workloads. These underutilized lines represent idle silicon that could instead be repurposed to absorb in-flight miss state, eliminating the need for dedicated miss handling storage entirely.

Several prior proposals have begun to explore this direction.

* The first two authors contributed equally to this work.

TABLE I
COMPARISON OF IN-SITU CACHE WITH PRIOR NON-BLOCKING CACHE DESIGNS ACROSS SYSTEM AND MICROARCHITECTURAL LEVELS.

	This Work	[17]	[11]	[16]	[18]	[19]	[8]	[9]
Application	Multicore cluster	Accelerator	GPU	CPU	GPGPU	Multicore cluster	GPU	Accelerator
Technology	ASIC P&R	FPGA	Simulator	RTL synth.	FPGA	RTL synth.	Silicon	Silicon
Miss Handling	In cache line	In cache line	Dedicated	Dedicated	Dedicated	Dedicated	Dedicated	Dedicated
Storage	In cache line	In cache line	Dedicated	Dedicated	Dedicated	Dedicated	Dedicated	Dedicated
Capacity	Dynamic	Static	Dynamic	Static	Static	Static	Static	Static
Write Buffer	In cache line	N.A.	N.A.	Dedicated	Write-through	N.A.	Dedicated	N.A.
System Organization	Hybrid	Single	Single	Single	Multi-Bank	Multi-Bank	Hybrid + MB	Hybrid

LLaMCAT [15] optimizes MSHR arbitration and thread throttling to reduce bandwidth contention from KV cache accesses, but leaves the structural area cost and scalability of dedicated miss handling storage unaddressed. Set-associative and cuckoo-hash-indexed Miss Status Handling Register (MSHR) organizations [10], [16] reduce associative lookup cost, but still require dedicated storage that grows with outstanding miss count. DL-MSHR [11] introduces a dynamically linked MSHR formed from a global pool, while MiCache [17] goes further by unifying cache and MSHR entries into a shared storage space. However, these proposals leave three key limitations unresolved. First, they focus on read-miss tracking and do not support write-miss handling, particularly sub-cacheline write coalescing, which is common in clusters with misaligned or sparse accesses from multiple processing elements (PEs). Second, miss-handling capacity remains inflexible: MiCache [17] allocates only a single cache line of subentry storage per MSHR entry, causing stalls when many read misses target the same address under long refill latencies. Third, prior proposals rely on architectural simulation and analytical modeling, with only partial synthesis of isolated components, leaving timing closure and true hardware cost not fully analyzed. Table I summarizes these limitations alongside system organization and validation level, positioning In-Situ Cache against representative prior designs across the cluster-cache landscape.

To address these limitations, we propose *In-Situ Cache*, a lightweight non-blocking cache that eliminates dedicated miss handling storage by repurposing underutilized cache lines to track outstanding misses in place. We integrate it into the open-source Spatz RISC-V Vector (RVV) cluster [1] as a runtime-reconfigurable hybrid SPM/cache L1, and demonstrate through full ASIC implementation in TSMC 7 nm.

Our main contributions are as follows:

- We propose the In-Situ Cache architecture, which repurposes underutilized cache resources for in-place miss handling through three key mechanisms: a unified in-cacheline MHA supporting both read and write misses, including sub-cacheline write coalescing; a Dynamic Subentry Expansion (DSE) mechanism that dynamically allocates multiple cache lines to a single miss entry without static provisioning; and a hash-based way-prediction scheme that reduces SRAM access energy proportionally to associativity.
- We integrate In-Situ Cache into the open-source Spatz cluster [1], replacing the L1 SPM with a runtime-

reconfigurable hybrid SPM/Cache subsystem that shares a unified SRAM array between software-managed and cache-managed operation.

- We evaluate In-Situ Cache in two configurations: backed by an on-chip L2 and by an off-chip DDR4-2400 channel, showing up to $2.96\times$ speedup on irregular workloads and $1.43\times$ on end-to-end sparse attention.
- We implement In-Situ Cache in TSMC 7 nm technology: a 128 KiB four-way configuration with hybrid SPM support occupies 461 kGE, with control logic accounting for only 8% of total area and consuming less than 0.05 W when executing workloads at 1 GHz with no frequency degradation with respect to the SPM-only baseline.

II. BACKGROUND AND MOTIVATION

A. Non-blocking Cache and Miss Handling Architectures

Caches in heterogeneous clusters and Machine Learning (ML) accelerators typically employ write-back with write-allocate policies [16], [19], [20] and require strong non-blocking capabilities to tolerate memory access latencies, whether from an on-chip L2 or off-chip DRAM. As shown in Figure 1, a conventional non-blocking cache includes not only data banks but also MSHRs and write buffers, together forming its MHA.

MSHRs track and merge outstanding read misses. They are typically organized as fully associative register files, each entry containing a tag field and a set of subentries. Upon a read miss, the cache checks the MSHRs for a matching entry. If one is found, the new miss is merged with the in-flight memory request by recording its metadata, such as the *PE id* and *reference id*, in a new subentry, avoiding redundant memory accesses. If no match exists, a new MSHR entry is allocated, and a memory request is issued. When the response arrives from memory, the data refills the corresponding cache line, and the subentries in the matching MSHR entry are used to generate responses to the requesters.

Write miss handling differs from read misses. Full-cacheline writes overwrite the line entirely and need no fetch, but sub-cacheline writes, which are common in clusters with misaligned or sparse accesses, must fetch the corresponding block and track partial modifications until it arrives. A write buffer in front of the cache coalesces sub-cacheline writes to reduce these fetches, and is typically implemented as a fully associative register file.

Both structures suffer from scalability issues. As reported in multiple papers [11], [17], the logic area occupied by MHA

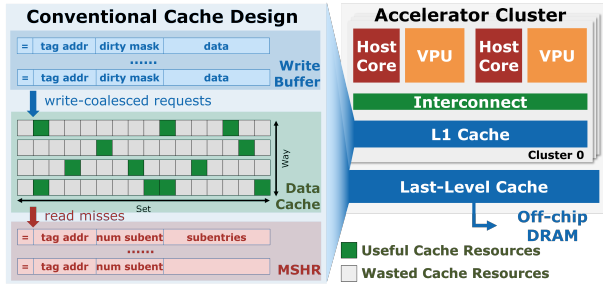


Fig. 1. Conventional cache controller design and its roles in heterogeneous clusters.

increases more than linearly, and hardly keeps pace with the increasing miss requirements from the workload.

In the remainder of this paper, we use MHA to refer collectively to MSHRs and write buffers.

B. Motivation of In-Situ Cache

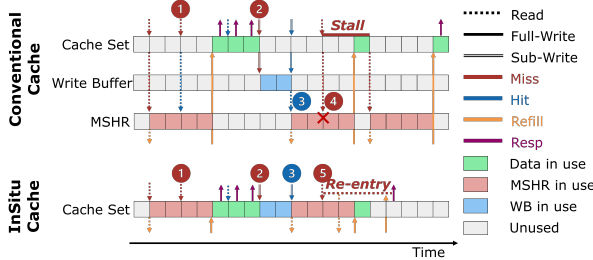


Fig. 2. Timing diagram comparing resource usage in conventional cache and In-Situ Cache. ① Read-miss MSHR-hit; ② Sub-cacheline write miss, filling write buffer; ③ Sub-cacheline hit in write buffer; ④ Read-miss MSHR-full stall in conventional cache; ⑤ Read-miss MSHR-full, re-entering a different way to avoid stalling in In-Situ Cache.

A timing diagram of typical traffic into a cache line is illustrated in Figure 2. The time a cache line holding useful data is only a small proportion of the overall period, and because of limited MHA resources, cache stalls occur repeatedly during read-heavy workloads. If wasted cache lines can be identified and repurposed for miss handling before they would otherwise be evicted, their unused portion of lifetime is shortened, reducing overall cache wastage while eliminating the area cost and scalability limitations of dedicated miss handling storage.

This observation motivates In-Situ Cache. However, realizing this concept raises three key challenges: (i) how to use cache lines to handle both read and write misses in a unified manner, (ii) how to provide sufficient storage for tracking miss metadata when many outstanding misses target the same cache block, and (iii) how to minimize power consumption without degrading the performance to make an energy-efficient system.

III. ARCHITECTURE

A. Unified In-Cacheline Miss Handling

In-Situ Cache extends the conventional *invalid/valid* cache line model with two additional states, *read-pend* and *write-pend*, to support in-place handling of read and write misses.

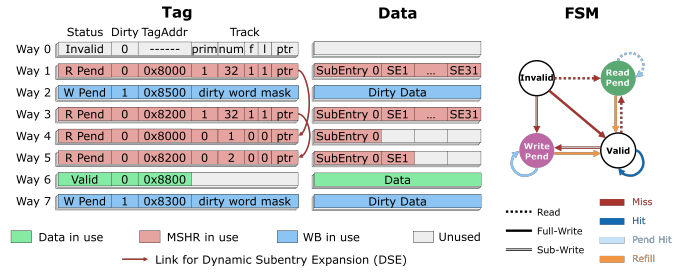


Fig. 3. Tag and data cache line design of In-Situ Cache and controller FSM status.

All four states participate in a unified tag lookup: a hit on a *read-pend* or *write-pend* line indicates that the new request can be merged with an already in-flight miss. To avoid confusion in the following discussion, we define *in-situ cache hit*, *in-situ read-pend hit*, *in-situ write-pend hit*, and *in-situ cache miss* to refer to hits on valid, read-pend, and write-pend lines, or a miss when none of these three states hit, respectively.

Figure 3 illustrates the cache line status transitions of In-Situ Cache. Upon an *in-situ cache miss* for a read or sub-cacheline write, an *invalid* or *valid* line in the same set is selected by the replacement policy and converted to *read-pend* or *write-pend* status. A memory request is issued to fetch the missing cache block, and the selected line is evicted first if it is dirty. Subsequent requests hitting an existing *read-pend* or *write-pend* line are merged with the in-flight miss. When data from main memory arrives, it is refilled into the corresponding pending line, with its status transitioning to *valid*, and cache responses are triggered if a *read-pend* line is refilled.

This in-cacheline miss-handling scheme introduces three potential stalling scenarios. *All-pend stalls* occur when every line in a set is in *read-pend* or *write-pend* status, preventing the acceptance of further *in-situ cache misses*. *Read-write conflict stalls* arise when a read request hits a *write-pend* line or a write request hits a *read-pend* line. *Subentry stalls* occur when a *read-pend* line runs out of subentry space, which is largely mitigated by the DSE mechanism described in Section III-B.

B. Cache Line Field Design

To support the unified in-cacheline miss handling scheme, In-Situ Cache augments the conventional cache line tag with an additional *track* field, as shown in Figure 3. The interpretation of the *track* and data fields depends on the line state.

For *read-pend* lines, the data field is divided into multiple subentries that store metadata of outstanding read misses. When a *read-pend* line runs out of subentry space, the DSE mechanism allocates a new *read-pend* line with the same tag address in the same set to track additional metadata. The *track* field of a *read-pend* line contains five subfields: *num*: subentries recorded; *full*: no remaining space; *prime*: primary line for the miss; *link* linked to an expansion; and *ptr*: pointer to the next linked line.

The pointer chain links all *read-pend* lines sharing the same tag address. When an *in-situ read-pend hit* occurs, new metadata is recorded only in the line whose *link* bit is clear, i.e., the

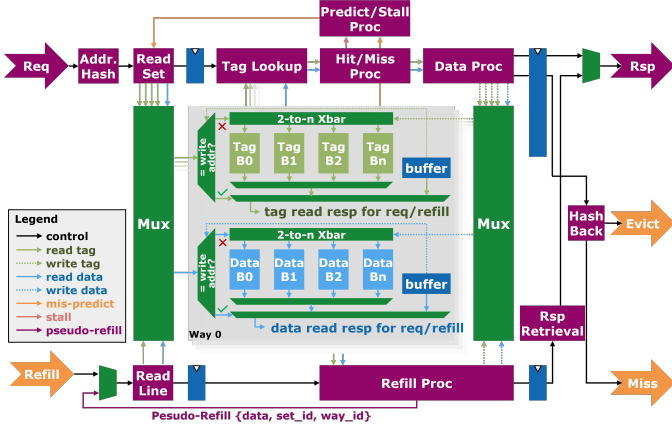


Fig. 4. Block diagram of In-Situ Cache controller and control pipelines.

tail of the chain. Allocating a new expanded *read-pend* line does not require a memory fetch, as no data is stored in the expansion. Upon refilling, the primary *read-pend* line receives the returned data and transitions to *valid*, while the remaining lines in the chain are sequentially invalidated by following the pointers. Cache responses are generated from the subentry metadata recorded across the chain during this process.

For *write-pend* lines, no subentries are needed because write requests are *fire-and-forget*: the requester does not wait for a response. The *track* field serves as a dirty-word bitmask recording which words have been modified, while the data field holds the partially updated write data. Upon refilling, the pending modifications are merged with the fetched cache block, and the line transitions to *valid*.

C. Data Path and Control Pipelines

At the microarchitectural level, In-Situ Cache uses a three-stage pipeline with two control paths: one for requests from the PEs and one for refills from memory, as illustrated in Figure 4.

In the request pipeline, the first stage hashes the incoming address to index a set and predict a target data way, using the hash-based scheme detailed in Section III-D, then issues read requests to all tag ways but only one predicted data way. The second stage performs tag comparison and data processing, including the prediction check. The third stage returns data to the requester on a hit or issues a memory request on a miss, evicting the replaced line first if it is dirty. The outgoing memory request carries the unhashed address along with the *set id* and *way id* of the pending cache line, so that the refill path can write the response directly to the correct location.

The refill pipeline handles data returned from memory. Using the *set id* and *way id* carried by the response, it writes directly to the target cache line. If the refilled line is a primary *read-pend* line with DSE expansions, the refill pipeline generates a *pseudo-refill* for each linked line following the pointer chain, sequentially invalidating all expanded *read-pend* lines. In the final stage, the refilled data and the subentry metadata collected across the chain are used to generate read responses to the PEs.

To meet the bandwidth requirements of the two control paths, the tag and data SRAMs are organized as pseudo-dual-port banks built from single-port macros. Read and write operations are interleaved across banks. When both target the same address, the write data is forwarded as the read response.

D. Way Prediction and Replacement Policy

In a conventional set-associative cache, every access reads all tag and data ways in the indexed set, and a replacement policy such as Least Recently Used (LRU) selects the victim on a miss. Two issues complicate replacement in In-Situ Cache: *read-pend* and *write-pend* lines must be excluded from LRU bookkeeping, and reading all data ways per access wastes SRAM read energy.

To address both issues, we designed a hash-based way-prediction scheme combined with pseudo-random replacement [21]. Because no per-line LRU state is maintained, the tag storage overhead and replacement logic complexity are both eliminated. On each access, a hash of the request address selects a single predicted way. Only the predicted data way is read alongside all tag ways in the first pipeline stage, reducing data SRAM read energy proportionally to the number of ways. The tag comparison in the second stage determines whether the prediction is correct. On a correct prediction, the access completes immediately with the data already read, moving to the third stage of the controller pipeline. On a misprediction, either due to a hit in a different way, or a miss where the predicted way is in a pending state, the pipeline replays from the first stage to access the correct way or an alternative non-pending way, described as *re-entry* in Figure 2. As a result, hit latency increases when a misprediction occurs. However, the occurrence of misprediction remains way below 1% in all benchmarks, and no visible performance drop is observed.

IV. SYSTEM INTEGRATION

We implemented In-Situ Cache in SystemVerilog and integrated it into the Spatz cluster [1], an open-source dual-core RVV heterogeneous cluster. Each Core Complex (CC) consists of a small scalar Snitch core and a 64-bit four-lane Spatz vector core, the latter accessing memory through a Vector Load/Store Unit (VLSU) that issues up to four 64-bit-wide word requests in parallel per cycle. CCs share a 128 KiB Tightly Coupled Data Memory (TCDM)-based L1 memory, organized as eight 64-bit-wide single-port SRAM banks interleaved at word granularity and connected to the VLSUs through a TCDM all-to-all crossbar. A cluster-local DMA engine, controlled by one CC, transfers data between the L1 SPM and the next level of the memory hierarchy.

In our design, the original L1 SPM is entirely replaced by In-Situ Cache, configured to the same 128 KiB capacity to maintain a comparable on-chip memory budget, as shown in Figure 5. A 512 bit Advanced eXtensible Interface (AXI) port connects the cache to the next level of the memory hierarchy.

A. Hybrid SPM/Cache

The hybrid SPM/Cache mode exposes the same physical SRAM array under two logical views, selected at runtime. This

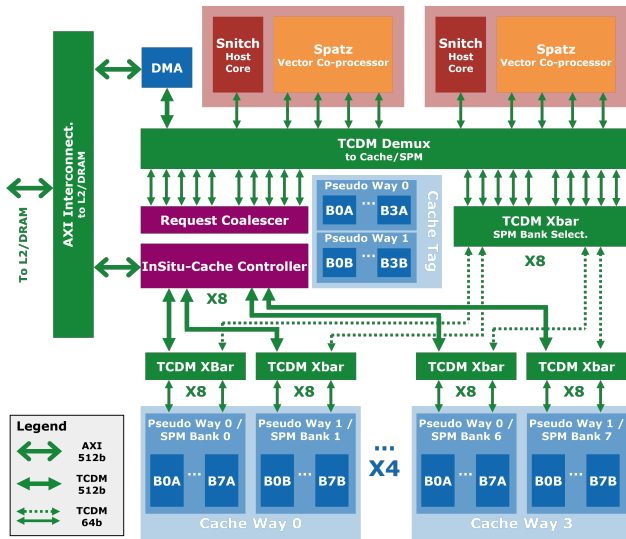


Fig. 5. Block diagram of In-Situ Cache integrated into Spatz cluster.

enables runtime selection of the most efficient mode for each kernel: SPM for regular streaming, cache for irregular accesses or mixed.

1) *Cache Mode*: From the cache perspective, the memory is organized as a four-way set-associative structure with a 512 bit cache-line width. Each cache way is internally split into two pseudo ways (A and B), which together emulate dual-port access using area-efficient single-port SRAMs. This pseudo-dual-port organization enables simultaneous tag and data reads and writes required by the In-Situ Cache miss-handling pipeline.

2) *SPM Mode*: From the SPM perspective, the same SRAM array is viewed as an eight-bank interleaved scratchpad, matching the baseline Spatz TCDM organization. Within each pseudo way, the eight constituent 64 bit-wide SRAM macros are individually addressable, providing the word-level access granularity and single-cycle latency expected by the VLSUs.

3) *Request Flow*: A TCDM demultiplexer at the top of the memory subsystem routes each VLSU request to either the cache path or the SPM path based on its address, with the boundary set by a programmable register. On the cache path, a request coalescer aggregates concurrent narrow requests that target the same cache line into a single 512 bit transaction before forwarding it to the In-Situ Cache controller. On the SPM path, requests bypass the cache controller and are routed in parallel through the TCDM crossbar directly to individual SRAM banks, preserving the deterministic latency of the baseline for non-conflicting parallel requests.

The boundary between the SPM and cache regions is runtime-configurable: a programmable memory-mapped register specifies how many cache sets are reserved as SPM, with the remaining capacity available to the cache. Resizing triggers an automatic cache flush to maintain correctness. When the SPM region is active, the cluster-local DMA engine handles bulk data transfers between the SPM and the next level of the memory hierarchy.

B. Testbed Environments

We evaluate In-Situ Cache in two system configurations, both operating at 1 GHz.

1) *Scenario A: L1 Cache with L2 Backing*: In this configuration, In-Situ Cache serves as the L1 data memory of the cluster. The AXI refill port is connected to a system-level L2 SPM, which we assume is preloaded with the working set before execution begins. This yields a fixed refill latency of 9 cycles, representative of a tightly-coupled on-chip L2, providing a maximum bandwidth of 64 Byte/cyc.

2) *Scenario B: Unified L1 and Last-Level Cache*: In this configuration, In-Situ Cache is connected directly to off-chip DRAM, modeled by the DRAMSys simulator [22] with a single-channel DDR4-2400 interface, providing a maximum bandwidth of 19.2 Byte/cyc. This setup exercises In-Situ Cache’s ability to tolerate long, variable refill latencies without dedicated miss handling storage, demonstrating its suitability as both an L1 and a lightweight LLC.

V. EVALUATION METHODOLOGY AND RESULTS

A. Evaluation Setup

We use Synopsys Fusion Compiler 2025.06 to synthesize, place, and route the Spatz cluster in TSMC 7 nm technology, targeting 65% area utilization on a 0.45 mm² die. All performance and Value Change Dump (VCD) files are collected with QuestaSim 2023.01 cycle-accurate simulator. Power analysis is performed using Synopsys PrimeTime 2022.03 on the post-layout netlist under typical operating conditions.

B. Physical Design

The timing of the design is closed at 1 GHz in the slow corner (SS/125C/0.675V). The critical path remains within the Snitch host core and is unaffected by the In-Situ cache integration. The fully routed design is shown in Figure 6.

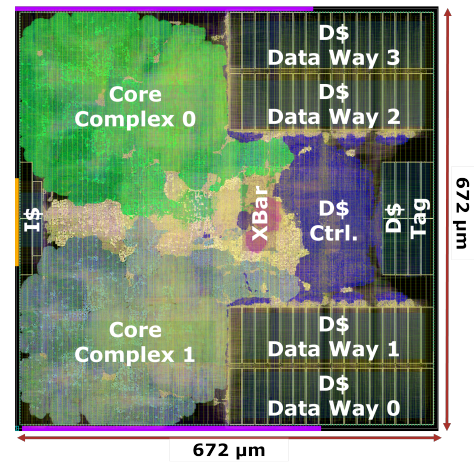


Fig. 6. Post P&R die shot of the Spatz cluster with hybrid In-Situ Cache as L1.

TABLE II

PERFORMANCE AND ENERGY EFFICIENCY COMPARISON BETWEEN SPM AND CACHE MODE OF CLUSTER. SPM-BASED KERNELS ARE IMPLEMENTED WITH DOUBLE-BUFFERING WHEN EXCEEDING 128 KiB. ALL METRICS (PERFORMANCE, POWER, ENERGY EFFICIENCY) ARE MEASURED AT 1 GHZ. PERFORMANCE IS NORMALIZED TO DOUBLE-PRECISION FLOP.

L1	Kernel	Size	FPU Util.	AI (L1-L2) [FLOP/B]	Perform. [GFLOPS]	Perform. Comp.	Power [mW]	En. Effic. [GFLOPS/W]	En. Effic. Comp.
Connect to L2 SPM (64 GB/s)									
SPM	GEMV-64b	128x128	35.67%	0.25	5.71		159	35.89	
Cache	GEMV-64b	128x128	46.46%	0.25	7.43	1.30×	155	47.96	1.34×
SPM	FFT-64b	2048	15.84%	0.55	2.53		87	29.13	
Cache	FFT-64b	2048	38.43%	0.55	6.15	2.43×	138	44.56	1.53×
SPM	GEMM-64b	64x64x64	98.51%	5.33	15.76		154	102.35	
Cache	GEMM-64b	64x64x64	97.45%	5.33	15.59	0.99×	164	95.07	0.93×
SPM	GEMM-64b	128x128x128	91.77%	10.67	14.68		154	95.35	
Cache	GEMM-64b	128x128x128	98.83%	10.67	15.81	1.08×	164	96.42	1.01×
SPM	VQ-GEMM-16b	8,8,2	77.70%	10.67	12.42		153	81.18	
Cache	VQ-GEMM-16b	8,8,2	54.22%	10.67	8.68	0.70×	149	58.26	0.72×
SPM	VQ-GEMM-16b	16,16,2	26.01%	10.67	4.16		113	36.81	
Cache	VQ-GEMM-16b	16,16,2	50.42%	10.67	8.07	1.94×	150	53.80	1.46×
SPM	sel-GEMV-16b	128x512	11.61%	0.25	1.86		131	14.18	
Cache	sel-GEMV-16b	128x512	26.42%	0.25	4.23	2.28×	158	26.75	1.89×
Connect to Offchip DDR4-2400 (19.2 GB/s)									
SPM	GEMV-64b	128x128	19.10%	0.25	3.06		142	21.52	
Cache	GEMV-64b	128x128	24.80%	0.25	3.97	1.30×	147	26.99	1.25×
SPM	FFT-64b	2048	10.26%	0.55	1.64		87	18.87	
Cache	FFT-64b	2048	30.38%	0.55	4.86	2.96×	138	35.22	1.87×
SPM	GEMM-64b	64x64x64	98.51%	5.33	15.76		154	102.35	
Cache	GEMM-64b	64x64x64	94.21%	5.33	15.07	0.96×	161	93.63	0.91×
SPM	GEMM-64b	128x128x128	89.79%	10.67	14.37		154	93.29	
Cache	GEMM-64b	128x128x128	90.96%	10.67	14.55	1.01×	161	90.40	0.97×
SPM	VQ-GEMM-16b	8,8,2	75.55%	10.67	12.09		152	79.54	
Cache	VQ-GEMM-16b	8,8,2	50.16%	10.67	8.02	0.66×	145	55.31	0.70×
SPM	VQ-GEMM-16b	16,16,2	17.31%	10.67	2.77		101	36.81	
Cache	VQ-GEMM-16b	16,16,2	30.40%	10.67	4.86	1.75×	119	40.84	1.49×
SPM	sel-GEMV-16b	128x512	10.60%	0.25	1.70		130	13.05	
Cache	sel-GEMV-16b	128x512	19.50%	0.25	3.12	1.84×	154	20.26	1.55×

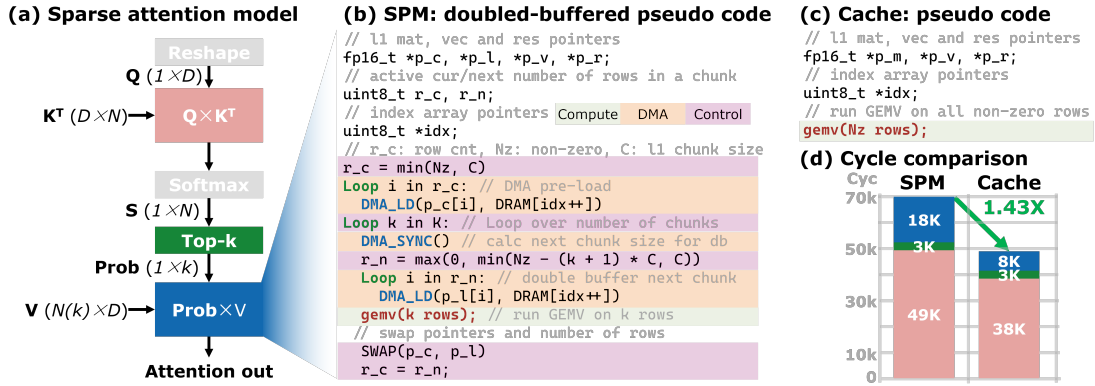


Fig. 7. (a) A simplified block diagram of the sparse attention model. Reshape and Softmax stages are not included in our kernel implementation; (b) Pseudo code of double-buffered selective GEMV for SPM; (c) Pseudo code of selective GEMV for cache; (d) Kernel cycle breakdown for SPM- and Cache-based variants.

C. Kernel Benchmark Setup and Analysis

We evaluate the two deployment scenarios from Section IV-B, using In-Situ Cache either as the cluster L1 data cache or as a unified L1/last-level cache. Across both scenarios, cache-mode execution is compared against the SPM-mode of the same cluster running explicitly software-managed kernels. This SPM-mode serves also as a tight proxy for the original SPM-only Spatz

cluster. In SPM-mode, requests bypass the cache controller through a combinational TCDM demultiplexer and reach the same multi-banked TCDM crossbar used in the baseline, yielding cycle-identical execution. The additional power overhead from the cache demux logic and leakage of the idle cache controller and tag SRAMs is characterized in Section V-E. For SPM kernels whose working sets exceed the 128 KiB L1-memory capacity, double buffering is employed to provide an optimized

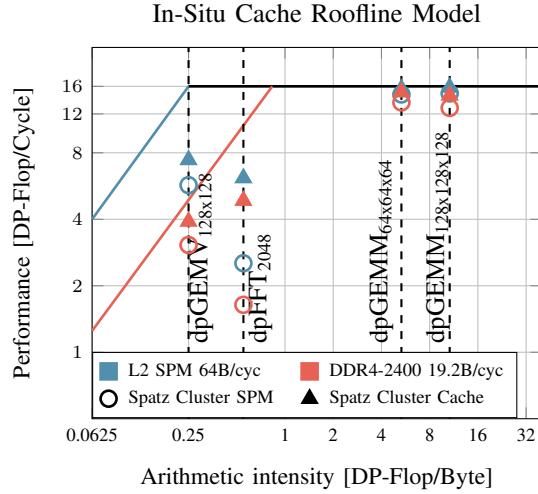


Fig. 8. The roofline analysis of Spatz cluster with hybrid In-Situ cache under two tested scenarios.

baseline. Table II summarizes the performance, power, and energy-efficiency results, while Figure 8 presents the roofline positioning of the evaluated micro-kernels.

1) *GEMV and GEMM*: Dense GEMV and General Matrix Multiply (GEMM) provide two regular-access baselines with contrasting data-reuse characteristics. Each matrix element in GEMV contributes to only one multiply-accumulate (MAC) operation, making execution memory-bound and highly sensitive to data-movement overheads. The initial data transfer in the double-buffered SPM baseline cannot be efficiently overlapped with computation, degrading performance. Our results show that cache-mode improves GEMV performance by $1.30\times$ in both deployment scenarios and improves energy efficiency by $1.25\text{--}1.34\times$, demonstrating that In-Situ Cache is effective for low-reuse, bandwidth-bound kernels.

In contrast, GEMM reuses operands across many MAC operations, making it compute-bound. Cache-mode matches the optimized SPM baselines on both GEMM sizes, confirming that In-Situ Cache preserves near-SPM throughput when data movement is hidden by computation. This confirms that In-Situ Cache can preserve near-SPM throughput for high-reuse dense kernels, where data-movement overheads are largely hidden by computation.

2) *FFT*: Fast Fourier Transform (FFT) represents a structured computation with non-contiguous data movement. Across stages, the bit-reversal permutation and stride changes between butterflies break contiguous reuse. As a result, software must repeatedly stage and reorganize data, exposing synchronization and data-movement overheads that are difficult to amortize due to the kernel’s relatively low arithmetic intensity.

Cache-mode improves FFT performance by $2.43\text{--}2.96\times$ and energy efficiency by $1.53\text{--}1.87\times$, capturing locality across stages without explicit data-movement orchestration in software.

3) *Vector-Quantized GEMM*: Vector-quantized GEMM reconstructs weight operands indirectly: the kernel loads compact indices and reads weight vectors from codebook entries before

issuing MAC operations. Denoting an AQLM [23] configuration as $VQ\langle d, b, r \rangle$ (d : vector dimension; 2^b : centroids per codebook; r : additive codebooks), we study two contrasting regimes: ① $VQ\langle 8, 8, 2 \rangle$ with an 8 KiB codebook that fits in L1 alongside computation tiles, and ② $VQ\langle 16, 16, 2 \rangle$ with a 4 MiB codebook that cannot.

The two regimes expose complementary strengths of the hybrid SPM/cache design. In regime ①, the codebook fits entirely in L1, and SPM mode stages it on chip with no explicit data movement from L2 or DRAM, attaining the higher throughput. Cache mode, on the same workload, exhibits lower FPU utilization, which we attribute to the mismatch between the fine granularity of the codebook lookups and the wider cache-line width. In regime ②, the relationship reverses: SPM mode must gather codebook entries explicitly, and the resulting scattered access pattern incurs extensive data-movement overhead. Cache mode instead captures the reused codebook entries dynamically within the L1 subsystem, delivering a $1.75\text{--}1.94\times$ speedup and a $1.46\text{--}1.49\times$ improvement in energy efficiency over SPM mode.

A single memory organization can hardly serve the contrasting access patterns across VQ-GEMM regimes. SPM mode exploits deterministic transfers when the codebook fits on chip, while cache mode wins once the codebook overflows the L1 and accesses become scattered. The hybrid design lets the same L1 subsystem serve both, with the mode selected at runtime to match the codebook footprint.

4) *Selective GEMV and Top-K Attention*: Selective GEMV represents the gathering phase of sparse attention, in which only the Value vectors corresponding to selected tokens are loaded and accumulated. Because the accessed rows are determined dynamically by the selected indices, the resulting memory accesses are irregular, making double-buffered SPM management complex and inefficient.

The benchmark results show that cache-based execution improves selective GEMV performance by $1.84\text{--}2.28\times$ and energy efficiency by $1.55\text{--}1.89\times$. This indicates that In-Situ Cache is particularly effective for sparse, data-dependent access patterns, where the cache hierarchy can capture reused data dynamically without software-managed gather operations.

To evaluate the impact on an end-to-end sparse-attention workload, we implement a 16-bit sparse attention layer consisting of three phases (Figure 7a): scoring with dense query-key GEMV, filtering with Radix Select, and gathering with selective GEMV over the Value matrix. The evaluated dimensions are 1×128 for the Query vector and 128×4096 for both the Key and Value matrices. Cache-based execution accelerates both the scoring and gathering phases, while the filtering phase remains dominated by scalar control overhead and shows limited sensitivity to the memory mode. Overall, In-Situ Cache achieves a $1.43\times$ end-to-end speedup while simplifying the implementation by avoiding explicit software-managed data movement, shown in Figure 7b–d.

D. Area Analysis

Figure 9b presents the logic area breakdown after routing. The two Snitch-Spatz CCs account for the largest share of the cluster area of 2.45 MGE. The data memory banks occupy 2 MGE, representing 34% of the total logic area. The In-Situ Cache controller requires only 461 kGE, composed primarily of the request coalescer and pipeline-stage First-In First-Outs (FIFOs). Together with the 243 kGE tag SRAMs, integrating In-Situ Cache into the cluster incurs approximately 700 kGE of area overhead, occupying 12% of the total cluster area.

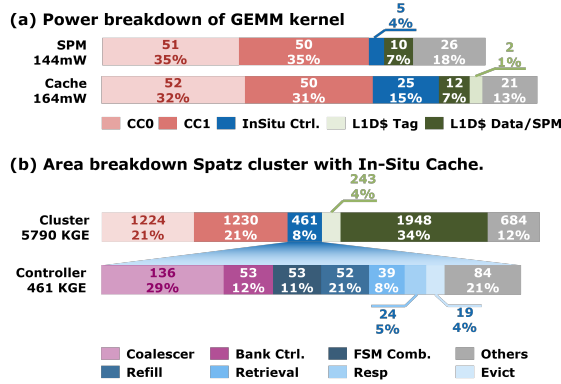


Fig. 9. (a) Power breakdown of Spatz cluster when running GEMM with data initialized in L2; (b) Area breakdown of Spatz cluster and detailed breakdown inside the In-Situ Cache controller.

E. Power Breakdown

Figure 9a provides a further breakdown for the GEMM-128x128x128 kernel. In this kernel, both the SPM and cache configurations achieve similar performance, providing a fair basis for comparison.

When running SPM-only workloads, which exercise the same request path as the original Spatz cluster, the cache integration incurs only 5 mW of additional static power from the idle cache controller and tag SRAMs, less than 4% of total power consumed by the cluster. In cache-managed workloads, the In-Situ Cache controller and tag SRAMs together consume only 27 mW to 40 mW more than the SPM configuration, contributing to less than 30% of total power spent, depending on the kernel. The hash-based way-prediction scheme significantly reduces SRAM access power by reading only a single data way per access, bringing the data bank power to the same level as the SPM implementation. Overall, In-Situ Cache incurs less than 15% energy efficiency degradation on regular kernels when connected to an L2 SPM, while achieving up to 2.28 $\times$ improvement on irregular workloads without requiring software-managed data orchestration. In-Situ Cache also delivers up to 2.96 $\times$ performance when directly connected to a long-latency DDR4-2400 channel, demonstrating its strong latency tolerance.

VI. CONCLUSION

We presented *In-Situ Cache*, a non-blocking cache that eliminates dedicated miss handling storage by repurposing underutilized cache lines for outstanding read and write misses.

Integrated as a runtime-configurable hybrid SPM/cache L1 in the open-source Spatz RVV cluster and implemented in TSMC 7 nm, it occupies only 8% of cluster area and sustains 102 GFLOPS/W on dense GEMM in SPM mode. In cache mode, it delivers up to 2.96 $\times$ speedup on FFT and 1.43 $\times$ on end-to-end sparse attention. On vector-quantized GEMM, the two modes prove complementary: SPM wins when the codebook fits on chip, cache wins once it overflows, and the hybrid design selects the right mode at runtime, without software-managed data orchestration.

REFERENCES

- [1] M. Perotti *et al.*, “Spatz: Clustering compact risc-v-based vector units to maximize computing efficiency,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 7, pp. 2488–2502, 2025.
- [2] S. Xu and C. Ramakrishnan, “Inside maia 100,” in *2024 IEEE Hot Chips 36 Symposium (HCS)*, 2024, pp. 1–17.
- [3] M. Ishii *et al.*, “Fast modular arithmetic on the kalray mppa-256 processor for an energy-efficient implementation of ecm,” *IEEE Transactions on Computers*, vol. 66, no. 12, pp. 2019–2030, 2017.
- [4] H. Zheng, K. Wang, and A. Louri, “A versatile and flexible chiplet-based system design for heterogeneous manycore architectures,” in *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2020, pp. 1–6.
- [5] H. Li *et al.*, “Pond: Cxl-based memory pooling systems for cloud platforms,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 574–587.
- [6] V. Egiastian *et al.*, “Extreme compression of large language models via additive quantization,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML’24. JMLR.org, 2024.
- [7] R. Child *et al.*, “Generating long sequences with sparse transformers,” 2019. [Online]. Available: <https://arxiv.org/abs/1904.10509>
- [8] NVIDIA. (2025) Rtx blackwell gpu architecture. [Online]. Available: <https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>
- [9] D. R. Ditzel and the Esperanto team, “Accelerating ml recommendation with over 1,000 risc-v/tensor processors on esperanto’s et-soc-1 chip,” *IEEE Micro*, vol. 42, no. 3, pp. 31–38, 2022.
- [10] M. Asiatici and P. Ienne, “Stop crying over your cache miss rate: Handling efficiently thousands of outstanding misses in fpgas,” in *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2019, pp. 310–319.
- [11] Y. Gu and L. Chen, “Dynamically linked mshrs for adaptive miss handling in gpus,” in *Proceedings of the 33rd International Conference on Supercomputing*, ser. ICS ’19. New York, NY, USA: Association for Computing Machinery, 2019, pp. 510–521.
- [12] M. Kharbutli and Y. Solihin, “Counter-based cache replacement and bypassing algorithms,” *IEEE Transactions on Computers*, vol. 57, no. 4, pp. 433–447, 2008.
- [13] J. Albericio *et al.*, “The reuse cache: Downsizing the shared last-level cache,” in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture*, 2013, pp. 310–321.
- [14] K. Kanellopoulos *et al.*, “Victima: Drastically increasing address translation reach by leveraging underutilized cache resources,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1178–1195.
- [15] Z. Zhou, C. Lai, and W. Zhang, “Llamcat: Optimizing large language model inference with cache arbitration and throttling,” in *Proceedings of the 54th International Conference on Parallel Processing*, ser. ICPP ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 460–469.
- [16] C. Fuguet, “Hpdcache: Open-source high-performance l1 data cache for risc-v cores,” in *Proceedings of the 20th ACM International Conference on Computing Frontiers*, 2023, pp. 377–378.
- [17] S. Xu *et al.*, “Micache: An mshr-inclusive non-blocking cache design for fpgas,” in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, ser. FPGA ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 22–32.

- [18] B. Tine *et al.*, “Vortex: Extending the risc-v isa for gpgpu and 3d-graphics,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 754–766.
- [19] M. R. Kakoei, V. Petrovic, and L. Benini, “A multi-banked shared-l1 cache architecture for tightly coupled processor clusters,” in *2012 International Symposium on System on Chip (SoC)*, 2012, pp. 1–5.
- [20] J. L. Hennessy and D. A. Patterson, *Computer Architecture, Fifth Edition: A Quantitative Approach*, 5th ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2011.
- [21] F. M. Sleiman, R. G. Dreslinski, and T. F. Wenisch, “Embedded way prediction for last-level caches,” in *2012 IEEE 30th International Conference on Computer Design (ICCD)*, 2012, pp. 167–174.
- [22] L. Steiner *et al.*, “Dramsys4.0: An open-source simulation framework for in-depth dram analyses,” *Int. J. Parallel Program.*, vol. 50, no. 2, p. 217–242, Apr. 2022.
- [23] V. Eghazarian *et al.*, “Extreme compression of large language models via additive quantization,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML’24, 2024.